UCLouvain

epl

École polytechnique de Louvain

[Titre principal du mémoire]

[Sous-titre éventuel]

Auteur: **[Prénom] [Nom]**
Promoteur: **[Prénom] [Nom]**
Lecteurs: **[Prénom] [Nom], [Prénom] [Nom]**
Année académique 2026–2027
[Intitulé officiel du diplôme de master]

Remerciements

Résumé

Abstract

Mots-clés

Déclaration d'utilisation de l'intelligence artificielle

Table des matières

1 Introduction **1**
1.1 Pourquoi ce travail . . . 1
1.2 Pour qui . . . 1
1.3 Comment . . . 2
1.4 Objectifs et questions de recherche . . . 3
1.5 Périmètre . . . 3
1.6 Plan du mémoire . . . 3

I État de l'art **5**

2 Apprendre la mécanique en première année **7**
2.1 Le cours de Physique 1 (BAC1 ingénieur civil) . . . 7
2.2 Conceptions erronées en dynamique . . . 7
2.3 Mesurer la compréhension conceptuelle . . . 8
2.4 Ce que cela demande à une animation . . . 8

3 Simulations interactives et apprentissage **9**
3.1 Effets des simulations sur l'apprentissage . . . 9
3.2 Le guidage . . . 9
3.3 Charge cognitive et principes multimédias . . . 10
3.4 Représentations multiples : schéma, formules, graphes . . . 10
3.5 Outils existants . . . 10

4 Ludification **13**
4.1 Définitions et mécanismes . . . 13
4.2 Effets mesurés . . . 13
4.3 Risques et conditions d'efficacité . . . 13

5 Calculer et afficher une simulation dans le navigateur **15**
5.1 Intégration des équations du mouvement . . . 15
5.2 Contraintes et dynamique basée sur les positions . . . 16
5.3 Systèmes de particules et fluides . . . 16
5.4 Précision, stabilité et validation . . . 16
5.5 Afficher : Canvas, SVG ou WebGL . . . 16
5.6 Bibliothèques JavaScript . . . 17

6 Intelligence artificielle et développement logiciel **19**
6.1 Assistants de programmation fondés sur des modèles de langage . . . 19
6.2 Les gains mesurés . . . 19
6.3 Fiabilité, vérification et responsabilité . . . 20
6.4 Usages en enseignement . . . 20

6.5 Synthèse de l'état de l'art : exigences pour le site 20

II Réalisation **23**

7 Cahier des charges et choix techniques **25**
7.1 Public visé et contraintes pédagogiques 25
7.2 Exigences fonctionnelles 25
7.3 Exigences techniques 25
7.4 Choix technologiques 25

8 Architecture du site **27**
8.1 Organisation du projet 27
8.2 Séparation calcul et affichage 27
8.3 Éléments visuels partagés 27
8.4 Mise en page et adaptation aux écrans 27

9 Exercices réalisés **29**
9.1 Mouvement d'un projectile 29
9.2 Pendule 29
9.3 Mouvement circulaire uniforme 29
9.4 Looping 29
9.5 Looping avec wagon : cercle et clothoïde 29

10 Mode jeu : le seau d'eau **31**
10.1 Principe et lien avec le looping 31
10.2 Modèle physique 31
10.3 Résolution numérique 31
10.4 Pilotage par le moteur 31
10.5 Défis toujours solubles 31

11 Méthodologie de développement assistée par IA **33**
11.1 Cadre et règles du projet 33
11.2 Cycle de développement 33
11.3 Agents de test et de vérification 33
11.4 Rôle de l'enseignant 33

12 Validation **35**
12.1 Vérification de la physique 35
12.2 Tests unitaires 35
12.3 Tests d'intégration sur plusieurs appareils 35
12.4 Erreurs détectées et corrigées 35

III Résultats **37**

13 Protocole d'évaluation **39**
13.1 Questions évaluées 39
13.2 Participants et déroulement 39
13.3 Instruments de mesure 39
13.4 Analyse des données 39
13.5 Aspects éthiques 39

14 Résultats quantitatifs **41**
14.1 Gains d'apprentissage 41
14.2 Utilisation du site 41
14.3 Mode jeu 41

15 Résultats qualitatifs **43**
15.1 Retours des étudiants 43
15.2 Retours des enseignants 43
15.3 Observations en classe 43

16 Discussion **45**
16.1 Réponses aux questions de recherche 45
16.2 Liens avec la littérature 45
16.3 Limites 45
16.4 Éthique, accessibilité et durabilité 45
16.5 Recommandations 45

17 Conclusion et perspectives **47**
17.1 Synthèse des contributions 47
17.2 Perspectives 47

Bibliographie **49**

A Formulaire du cours **53**

B Questionnaires **55**

C Liste des exercices et du code **57**

D Détails des tests **59**

Table des figures

1.1 Une page du site (looping avec wagon). À gauche, la scène avec les forces ; au centre, le diagramme du corps libre et les énergies ; à droite, les paramètres et un graphe en fonction du temps ; en bas, les formules avec leurs valeurs substituées. Toutes les vues suivent le même instant de la simulation. 2

Liste des tableaux

3.1 Adéquation des outils existants à un cours précis. « À construire » : possible, mais l'enseignant doit le programmer lui-même. 11

6.1 Exigences tirées de l'état de l'art et réponse du site. 21

Chapitre 1

Introduction

1.1 Pourquoi ce travail

En première année d'université, la mécanique est souvent la première rencontre des étudiants avec une physique formalisée : des lois écrites sous forme vectorielle, appliquées à des systèmes qui évoluent dans le temps. Beaucoup apprennent à appliquer les formules sans se représenter ce qu'elles décrivent. Un étudiant peut calculer correctement l'accélération d'un objet en mouvement circulaire et penser malgré tout qu'une « force centrifuge » le pousse vers l'extérieur. Ces idées issues du sens commun sont connues depuis longtemps et résistent à un enseignement classique [1]. Les cours où l'étudiant est actif et confronte ses idées à ce qui se passe réellement donnent de bien meilleurs résultats [2] (chapitre 2).

Une partie du problème tient aux supports. Un schéma au tableau ou une figure du syllabus est immobile : il montre le système à un instant, pas la façon dont la vitesse, l'accélération et les forces évoluent pendant le mouvement. Une animation interactive comble ce manque. L'étudiant voit l'objet bouger, les vecteurs changer de direction et de longueur, et peut modifier un paramètre pour observer son effet. Des simulations conçues de cette façon ont montré leur intérêt pour l'apprentissage [3], [4] (chapitre 3).

Produire de telles animations reste pourtant coûteux. Il faut maîtriser la physique, pour que le modèle soit juste, et la programmation d'interfaces, pour qu'il soit lisible et manipulable. C'est ce coût que l'arrivée de l'intelligence artificielle générative remet en question. Depuis 2021, des modèles de langage entraînés sur du code écrivent une fonction à partir de sa description [5], et les assistants actuels lisent un projet entier, le modifient, lancent les tests et corrigent leurs erreurs (chapitre 6). Une animation qui demandait des semaines de développement devient envisageable en quelques jours. Ce mémoire part de la rencontre de ces deux constats : les étudiants ont besoin de *voir* la physique, et l'IA pourrait rendre possible de la leur montrer à moindre coût [6].

1.2 Pour qui

Le public visé en premier lieu est celui des étudiants de première année du cours de Physique 1 (LEPL1201) de l'École polytechnique de Louvain. Les animations reprennent le formulaire et les notations de ce cours [7], pour que l'étudiant y retrouve exactement les symboles qu'il rencontre au cours et à l'examen. Le cours de physique de la Faculté des sciences de la motricité (LFSM1105), qui partage la même mécanique, peut utiliser les mêmes pages.

L'objectif pour l'étudiant est de faciliter la compréhension en *montrant* ce que les formules calculent. Chaque animation affiche le phénomène, toutes les forces qui agissent avec leur direction et leur grandeur, et les formules du cours écrites trois fois : sous forme symbolique,

avec les valeurs numériques du moment remplacées, puis avec le résultat (figure 1.1). L'étudiant peut ainsi suivre le lien entre le calcul qu'il ferait sur papier et ce qu'il voit à l'écran.



FIGURE 1.1 – Une page du site (looping avec wagon). À gauche, la scène avec les forces ; au centre, le diagramme du corps libre et les énergies ; à droite, les paramètres et un graphe en fonction du temps ; en bas, les formules avec leurs valeurs substituées. Toutes les vues suivent le même instant de la simulation.

Le travail s'adresse aussi aux enseignants. Ce sont eux qui utilisent les animations au cours ou les proposent en travail autonome, et ce sont eux qui devraient pouvoir en créer de nouvelles pour leurs propres exercices. Le site du cours propose déjà deux animations [8] ; la question est de savoir comment en produire davantage, sans y consacrer un temps démesuré et sans sacrifier la justesse.

1.3 Comment

Nous avons développé un site de simulations de mécanique en nous faisant assister par l'IA, principalement Claude Code (Anthropic) et ChatGPT Codex (OpenAI). L'enjeu n'est pas de « faire écrire du code » par l'IA. Laissé sans cadre, un assistant produit un code qui a l'air juste mais peut contenir une formule fausse, une hypothèse cachée ou une structure impossible à faire évoluer ; une part importante du code généré contient par exemple des failles que l'utilisateur ne voit pas [9]. En physique, une animation fausse est d'autant plus dangereuse qu'elle peut avoir l'air juste.

Nous avons donc encadré l'IA pour obtenir trois propriétés à la fois :

— **une physique correcte** : les formules exactes du cours sont utilisées quand elles existent, les hypothèses du modèle sont écrites sur la page, et chaque calcul est vérifié par des tests et par un agent séparé qui redérive les formules et les recoupe numériquement ;

— **un code maintenable** : tous les exercices suivent la même structure (un fichier de calcul, un fichier d'affichage, une page), les calculs physiques ne sont jamais partagés

entre exercices pour rester faciles à vérifier, et le site n'utilise aucune bibliothèque externe ;

— **une création facilitée et en partie automatisée** : les règles du projet sont écrites une seule fois dans un fichier lu par l'assistant à chaque session, et des procédures réutilisables (*skills*) et des agents spécialisés prennent en charge les tâches récurrentes : créer un exercice, écrire les tests, vérifier la physique, contrôler qu'un exercice est terminé.

Les choix pédagogiques, la validation des résultats et la responsabilité de leur exactitude restent les nôtres. L'IA accélère le travail, le cadre en garantit la qualité, et ce mémoire évalue les deux. Le site est publié en ligne [10].

1.4 Objectifs et questions de recherche

Le sujet fixe deux objectifs [6]. Le premier est de développer un ensemble d'animations JavaScript interactives pour la physique de première année, physiquement justes et utiles en classe. Le second est d'évaluer de façon critique ce que les outils d'IA apportent au programmeur-concepteur, pour écrire le code comme pour organiser l'application. Nous en tirons trois questions :

1. **Conception.** Quelles caractéristiques d'une animation aident un étudiant de première année à relier le phénomène, les forces et les formules du cours ?
2. **Développement.** Des enseignants qui programment peuvent-ils produire de telles animations avec un assistant d'IA, avec une physique vérifiée et un code maintenable ? Que gagnent-ils en temps, et à quelle étape ?
3. **Limites.** Où l'assistant se trompe-t-il, et quelles vérifications faut-il pour qu'une erreur n'atteigne pas les étudiants ?

L'état de l'art répond en grande partie à la première question et prépare les deux autres, qui sont traitées dans les parties Réalisation et Résultats.

1.5 Périmètre

Le site compte aujourd'hui cinq exercices de mécanique : le tir parabolique, le pendule, le mouvement circulaire uniforme et deux versions du looping. Quatre d'entre eux proposent un mode jeu expérimental, dans lequel l'étudiant doit atteindre un objectif en réglant les conditions initiales ; dans les deux versions du looping, il s'agit de faire tourner un seau d'eau sans le renverser. Le sujet couvre aussi l'électricité élémentaire, que le formulaire du cours aborde (force de Coulomb, théorème de Gauss, potentiel électrique) [7]. [À COMPLÉTER : exercices d'électricité ajoutés d'ici le dépôt, ou électricité hors périmètre, et pourquoi.]

1.6 Plan du mémoire

Le mémoire est organisé en trois parties.

La **partie I**, *État de l'art*, rassemble ce que l'on sait déjà. Le chapitre 2 résume les difficultés des étudiants en mécanique. Le chapitre 3 présente ce que la recherche montre de l'apprentissage avec des simulations, ainsi que les outils existants. Le chapitre 4 traite des éléments de jeu dans l'apprentissage. Le chapitre 5 passe en revue les méthodes de calcul et d'affichage utilisables dans un navigateur. Le chapitre 6 fait le point sur les assistants de programmation et se termine par les exigences que nous en tirons pour le site.

La **partie II**, *Réalisation*, décrit ce que nous avons construit : nos outils et nos choix techniques (chapitre 7), l'architecture du site (chapitre 8), les exercices (chapitre 9), le mode

jeu du seau d'eau (chapitre 10), notre façon d'utiliser l'IA avec ses règles, skills et agents (chapitre 11) et la validation de la physique et du code (chapitre 12).

La **partie III**, *Résultats*, présente la façon dont nous évaluons le travail (chapitre 13), les résultats mesurés, dont la comparaison entre la théorie et la simulation et l'apport réel de l'IA (chapitre 14), les retours des utilisateurs (chapitre 15) et leur discussion (chapitre 16). La conclusion (chapitre 17) répond aux trois questions et ouvre sur la suite.

Première partie

État de l'art

Chapitre 2

Apprendre la mécanique en première année

Ce chapitre résume ce que la recherche en didactique de la physique sait des difficultés des étudiants en mécanique. Il en tire les premières exigences pour nos animations.

2.1 Le cours de Physique 1 (BAC1 ingénieur civil)

Le cours LEPL1201 couvre la mécanique du point et du solide, puis l'électricité élémentaire. Son formulaire, autorisé à l'examen, fixe le contenu et les notations [7] :

- la cinématique, dont le mouvement d'un projectile écrit composante par composante : $x(t) = u_0 t + x_0$ et $y(t) = -gt^2/2 + v_0 t + y_0$, avec le vocabulaire du mouvement rectiligne uniforme (MRU) et uniformément accéléré (MRUA) ;
- le mouvement circulaire, avec $v = r\omega$, l'accélération radiale $a_r = -r\omega^2$ et tangentielle $a_\theta = r\alpha$;
- la dynamique (lois de Newton, moment cinétique) et le théorème de l'énergie mécanique ;
- la force de gravitation et la force électrique, le théorème de Gauss et le potentiel électrique.

Ces notations comptent pour une animation. Un étudiant qui voit v_{0x} dans une simulation et u_0 au cours doit faire une traduction de plus, qui n'apporte rien et peut l'induire en erreur. Toutes nos pages reprennent donc les symboles du formulaire.

2.2 Conceptions erronées en dynamique

Les étudiants n'arrivent pas en mécanique sans idées. Ils ont une « physique du sens commun », construite par l'expérience quotidienne, et elle contredit souvent les lois de Newton. Halloun et Hestenes l'ont mesurée chez des étudiants d'université avant et après un cours classique : beaucoup pensent qu'un mouvement exige une force dans le sens de la vitesse, et cette idée change peu après le cours [1]. Viennot avait observé le même raisonnement chez des étudiants de plusieurs pays [11]. On retrouve dans ces travaux, et dans l'expérience courante des enseignants, les conceptions suivantes :

- une force est nécessaire pour entretenir le mouvement, et elle pointe dans le sens de la vitesse ;
- la vitesse et l'accélération ont la même direction, en particulier dans un virage ;
- dans un mouvement circulaire, une « force centrifuge » pousse l'objet vers l'extérieur ;
- la réaction normale est toujours égale au poids.

Le mouvement circulaire réunit plusieurs de ces difficultés. L'accélération pointe vers le centre alors que la vitesse est tangente, ce qui contredit l'idée que l'accélération suit la vitesse. Il n'existe pas de « force centripète » qui s'ajouterait aux autres : c'est la résultante du poids, de la réaction normale ou de la tension qui joue ce rôle.

Le looping en est l'exemple classique. Prenons un wagon assimilé à un point, lâché sans vitesse à la hauteur h au-dessus du bas d'un looping circulaire de rayon R, sans frottement. Pour savoir s'il fait le tour, il faut combiner deux principes. La conservation de l'énergie donne sa vitesse au sommet, $v_s^2 = 2g(h - 2R)$. La deuxième loi de Newton selon le rayon donne alors la réaction du rail au sommet, $N_s = m(v_s^2/R - g)$, qui doit rester positive ou nulle : sinon, le rail devrait tirer le wagon, ce qu'il ne peut pas faire. On obtient la hauteur minimale $h_{\min} = 5R/2$. Le seau d'eau que l'on fait tourner dans un plan vertical obéit à la même condition, avec la tension de la corde à la place de la réaction normale. Les vrais loopings ne sont d'ailleurs pas circulaires, pour limiter l'accélération subie par les passagers [12], ce qui ouvre l'exercice sur une situation réelle.

2.3 Mesurer la compréhension conceptuelle

Pour mesurer ces conceptions, Hestenes, Wells et Swackhamer ont construit le *Force Concept Inventory* (FCI) [13]. C'est un questionnaire à choix multiples sans calcul, dont chaque mauvaise réponse correspond à une conception erronée précise. Il est devenu l'instrument de référence pour comparer des méthodes d'enseignement de la mécanique.

Hake l'a utilisé sur plus de six mille étudiants, dans des cours d'écoles secondaires, de collèges et d'universités [2]. Pour comparer des groupes qui partent de niveaux différents, il utilise le *gain normalisé* : la part de ce qui restait à apprendre qui a été acquise pendant le cours,

$$\langle g \rangle = \frac{\text{score final} - \text{score initial}}{100\,\% - \text{score initial}}.$$

Ce gain vaut en moyenne 0,23 dans les cours magistraux classiques et 0,48 dans les cours à « engagement interactif », où l'étudiant est actif et reçoit un retour immédiat sur ses raisonnements. Le constat est net : un cours magistral, même bien donné, corrige peu les conceptions erronées ; il faut que l'étudiant confronte ses idées à ce qui se passe réellement.

Ce type de mesure servira à évaluer nos animations si un test avec des étudiants est possible (chapitre 13).

2.4 Ce que cela demande à une animation

Ces travaux donnent trois exigences directes :

— montrer *toutes* les forces, avec leur direction et leur grandeur, pour que l'étudiant voie qu'aucune force centrifuge n'apparaît, et que la réaction normale varie ;

— montrer la vitesse et l'accélération séparément, pour que leur différence de direction soit visible ;

— laisser l'étudiant prédire puis vérifier : changer un paramètre, voir si le wagon fait le tour, et comprendre pourquoi.

À ces exigences s'ajoute celle de la section précédente : parler la langue du cours, avec ses notations.

Chapitre 3

Simulations interactives et apprentissage

Ce chapitre présente ce que la recherche montre de l'apprentissage avec des simulations, les conditions de leur efficacité, puis les outils qui existent déjà.

3.1 Effets des simulations sur l'apprentissage

Les simulations aident à apprendre, à condition d'être bien employées. La revue de Rutten, van Joolingen et van der Veen, qui couvre une décennie d'études en sciences, conclut que les simulations améliorent l'enseignement classique quand elles le complètent, en particulier pour préparer ou accompagner une activité pratique [4]. Plus largement, la méta-analyse de Freeman et al. montre que l'apprentissage actif, dont les simulations sont une forme possible, améliore les résultats aux examens et réduit l'échec en sciences et en ingénierie [14].

Le projet PhET, à l'université du Colorado, est le plus connu du domaine. Ses simulations ont été conçues puis testées en entretiens avec des étudiants. Les auteurs en tirent quelques principes : une réponse immédiate à chaque action, plusieurs représentations du même phénomène, et une exploration laissée ouverte mais orientée par l'interface [3], [15]. Finkelstein et al. ont même montré que, pour les circuits électriques, des étudiants qui avaient travaillé avec une simulation réussissaient mieux à monter un vrai circuit que ceux qui avaient manipulé du vrai matériel [16]. Une simulation peut en effet montrer ce qu'un laboratoire cache : le courant, les forces, le champ.

3.2 Le guidage

Laisser l'étudiant découvrir seul ne suffit pas. De Jong et van Joolingen ont passé en revue l'apprentissage par découverte avec des simulations : les étudiants formulent mal leurs hypothèses, changent plusieurs variables à la fois et tirent des conclusions de trop peu d'essais [17]. La découverte devient efficace quand l'environnement guide : hypothèses à tester proposées, une seule variable modifiée à la fois, un objectif à atteindre.

Pour une animation, cela se traduit par un verdict clair (le wagon fait le tour, ou décolle à tel angle) et par des questions ou des défis ciblés. Afficher les hypothèses du *modèle* (frottements négligés, objet assimilé à un point, corde inextensible) est une exigence voisine mais distincte : l'étudiant doit savoir ce que la simulation idéalise pour juger ce qu'elle montre.

3.3 Charge cognitive et principes multimédias

Une simulation riche peut aussi gêner. La théorie de la charge cognitive part d'un fait : la mémoire de travail est limitée, et tout ce qui l'occupe sans servir l'apprentissage le ralentit [18]. Mayer en a tiré des principes de conception vérifiés expérimentalement [19]. Trois concernent directement une page de simulation :

— *contiguïté* : un texte et la figure qu'il commente doivent être proches dans l'espace et dans le temps, pour que l'étudiant n'ait pas à garder l'un en mémoire en cherchant l'autre ;

— *signalement* : des repères visuels constants indiquent ce qui compte, par exemple une couleur fixe par type de force ;

— *cohérence* : ce qui n'apporte rien à la compréhension (décor, effets) doit être retiré.

Ces principes justifient plusieurs de nos choix : tout tient sur un écran quand c'est possible, chaque force a sa couleur dans tous les exercices (poids en rouge, réaction normale en vert, tension en violet…), et les formules sont placées juste sous la scène.

3.4 Représentations multiples : schéma, formules, graphes

En physique, le même phénomène se lit sous plusieurs formes : la scène, le diagramme du corps libre, l'équation, le graphe en fonction du temps. Ainsworth montre que ces représentations multiples aident quand elles se complètent ou s'éclairent l'une l'autre, mais que passer de l'une à l'autre est une tâche en soi, que les débutants réussissent mal sans aide [20].

Une animation peut aider à faire ce lien : toutes les vues avancent au même instant, et la formule affiche les valeurs du moment présent. C'est l'idée de la formule écrite trois fois (symboles, valeurs substituées, résultat) : elle relie le calcul de l'exercice à ce que montre la scène. Ce lien automatique a un revers : il peut dispenser l'étudiant de faire la traduction lui-même [20]. D'où l'intérêt d'activités où il doit prédire avant de voir, comme les défis du mode jeu (chapitre 4).

3.5 Outils existants

De nombreux outils proposent déjà des simulations de mécanique :

— **PhET** : simulations en HTML5, gratuites et traduites, conçues pour un public large, du secondaire à l'université [21] ;

— **Easy Java/JavaScript Simulations (EjsS)** : un environnement qui permet à un enseignant de construire une simulation à partir de ses équations, sans écrire toute l'interface [22] ;

— **myPhysicsLab** : des simulations de mécanique à code ouvert, avec intégration numérique explicite et graphes [23] ;

— **GeoGebra** : un logiciel de géométrie dynamique dans lequel un enseignant construit lui-même ses animations [24] ;

— **les animations du cours LEPL1201** : « Vitesse et accélération » et « Comète de Halley », écrites en JavaScript pour le cours [8].

Le tableau 3.1 compare ces familles d'outils sur les besoins propres à un cours. Il ne juge pas leur qualité générale, qui est bonne, mais leur adéquation à LEPL1201.

À part les animations du cours, aucun de ces outils n'est écrit pour un cours précis. Les simulations générales n'emploient pas les notations du formulaire, n'affichent pas le calcul de l'exercice avec ses valeurs, et sont difficiles à modifier par un enseignant. Les outils de construction peuvent être adaptés, mais au prix du temps de développement que nous

Tableau 3.1 – Adéquation des outils existants à un cours précis. « À construire » : possible, mais l'enseignant doit le programmer lui-même.

	Simulations générales (PhET, myPhysicsLab)	Outils de construction (EjsS, GeoGebra)	Animations du cours (LEPL1201)	Notre site
Notations du formulaire	non	à construire	oui	oui
Calcul avec valeurs substituées	non	à construire	non	oui
Forces et diagramme du corps libre	selon la simulation	à construire	$\vec{v}$, $\vec{a}$ seulement	oui
Nombre de situations	grand	–	2	5
Effort pour un nouvel exercice	–	élevé	élevé	à mesurer

cherchons justement à réduire. Notre travail ne cherche donc pas à refaire PhET : il vise des animations écrites pour un cours, dans ses notations, qu'un enseignant peut produire et modifier lui-même. La dernière ligne du tableau est une hypothèse, que la partie Résultats doit vérifier.

Chapitre 4

Ludification

Quatre de nos exercices proposent un mode jeu. Ce chapitre situe ce choix dans la littérature : ce qu'est la ludification, ce qu'on en a mesuré, et à quelles conditions elle sert l'apprentissage plutôt que le score.

4.1 Définitions et mécanismes

La ludification (*gamification*) consiste à utiliser des éléments de conception issus des jeux dans un contexte qui n'est pas un jeu [25]. Elle se distingue du jeu sérieux, qui est un jeu complet conçu dans un but autre que le divertissement. Notre mode jeu est à la frontière : il ajoute des éléments de jeu à un exercice existant, sans devenir un jeu autonome.

Les mécanismes les plus courants sont :

— un **objectif** clair, que l'on atteint ou non ;

— une **rétroaction** immédiate, qui dit si l'essai a réussi ;

— une **progression** par paliers, qui donne un but à plus long terme ;

— un **défi** adapté, ni trivial ni impossible.

4.2 Effets mesurés

Les revues de la littérature trouvent des effets plutôt positifs, mais variables. Hamari, Koivisto et Sarsa ont passé en revue les études empiriques sur la ludification : la plupart rapportent des effets positifs, qui dépendent fortement du contexte et des utilisateurs [26]. La méta-analyse de Sailer et Homner, centrée sur l'apprentissage, trouve des effets positifs faibles à moyens sur les résultats d'apprentissage, la motivation et le comportement des apprenants, avec une grande variabilité entre études [27]. La ludification n'est donc ni inutile ni efficace par principe : son effet dépend de la façon dont elle est conçue.

4.3 Risques et conditions d'efficacité

Le risque principal est que l'étudiant optimise le score sans comprendre, par essais et erreurs. Un jeu où l'on gagne en déplaçant un curseur jusqu'à ce que ça marche n'apprend rien de la physique. Deux conditions limitent ce risque.

La première est que la règle du jeu soit la relation physique elle-même. Dans le jeu du seau d'eau, le seau doit passer au point bas à une vitesse $v_b \geq \sqrt{5gR}$, où R est le rayon du cercle décrit, pour que l'eau reste au fond au sommet ; et la tension au point bas, $m(v_b^2/R + g)$, ne doit pas dépasser la tension de rupture de la corde. Gagner revient à appliquer exactement les deux conditions du cours.

La seconde est de cacher ce qui donnerait la réponse : en mode jeu, les formules, les graphes et la trajectoire prévue sont masqués, et tout changement de paramètre remet la simulation à zéro pour qu'on ne puisse pas viser en regardant une trajectoire à moitié tracée. Chaque défi est tiré au hasard ; dans le jeu du seau, le tirage porte sur la grandeur à régler et sur le lieu, donc sur la valeur de g. On ne peut donc pas retenir une réponse d'un essai à l'autre.

Ces précautions diminuent le tâtonnement sans le supprimer : les curseurs permettent encore de chercher la bonne valeur par essais successifs. Le mode jeu ne remplace donc pas l'exercice ; il le prolonge pour l'étudiant qui a compris la relation et veut la mettre à l'épreuve. Mesurer s'il est utilisé ainsi fait partie de l'évaluation (chapitre 13).

Un dernier choix concerne la sobriété. Le principe de cohérence de Mayer [19] invite à retirer ce qui distrait. Nous avons privilégié une physique plus réaliste (corde souple, eau en gouttes qui sortent du seau) plutôt que des effets décoratifs : ce qui rend le jeu intéressant doit être le phénomène lui-même.

Chapitre 5

Calculer et afficher une simulation dans le navigateur

Une animation doit donner, à chaque image, la position et la vitesse du système, puis les dessiner. Ce chapitre passe en revue les méthodes de calcul et d'affichage disponibles dans un navigateur, et ce qu'elles impliquent pour un outil d'enseignement.

5.1 Intégration des équations du mouvement

Deux voies existent. Quand le cours fournit une solution exacte, il suffit de l'évaluer à l'instant t. Pour le tir parabolique, $x(t) = u_0 t + x_0$ et $y(t) = -gt^2/2 + v_0 t + y_0$: le résultat est alors exactement celui que l'étudiant calcule à la main, à l'arrondi près. Quand il n'y a pas de solution simple, comme pour le pendule aux grands angles ou un wagon sur un rail de forme quelconque, il faut intégrer l'équation du mouvement pas à pas. La simulation ne donne alors plus la valeur exacte, et l'écart avec la théorie doit être mesuré.

Le choix du schéma d'intégration décide de la précision et de la stabilité [28], [29]. Les plus utilisés en simulation sont :

— **Euler explicite** : la position avance avec l'ancienne vitesse. C'est le plus simple, mais l'erreur est d'ordre 1 en Δt (diviser le pas par deux ne divise l'erreur que par deux), et l'énergie d'un oscillateur augmente à chaque période : un pendule simulé ainsi finit par faire des tours complets ;

— **Euler semi-implicite** et **Verlet** : la position avance avec la vitesse déjà mise à jour, ou l'on travaille directement sur les positions successives. Ces schémas sont dits symplectiques : l'énergie oscille légèrement autour de sa vraie valeur au lieu de dériver [29], [30]. Jakobsen a popularisé Verlet dans le jeu vidéo [31] ;

— **Runge–Kutta d'ordre 4 (RK4)** : quatre évaluations de l'accélération par pas, une erreur d'ordre 4 (diviser le pas par deux divise l'erreur par seize). Pour un pas donné, il est bien plus précis, ce qui convient quand la simulation doit coller aux valeurs du cours [28]. Il n'est pas symplectique : sur de très longues durées, l'énergie dérive lentement [29].

Un point pratique s'ajoute. Le navigateur affiche environ soixante images par seconde, mais cette fréquence varie d'un appareil à l'autre. Si le pas de calcul suit l'image, le résultat dépend de l'appareil. La solution usuelle est de calculer avec un pas fixe, plusieurs fois par image si nécessaire, indépendamment de l'affichage [32].

5.2 Contraintes et dynamique basée sur les positions

Certains systèmes ont des liaisons : une corde qui ne s'allonge pas, une bille qui reste sur un rail, de l'eau qui ne traverse pas le fond d'un seau. Les écrire comme des forces de ressort très raides rend le calcul instable, sauf avec un pas très petit.

L'informatique graphique a développé une autre voie, la dynamique basée sur les positions (*Position Based Dynamics*, PBD) [31], [33], [34]. À chaque pas, on avance les particules librement, puis on corrige leurs positions pour respecter les liaisons, et l'on en déduit les vitesses. La méthode est stable et simple, mais la raideur obtenue dépend du nombre d'itérations et du pas. Sa version étendue (XPBD) corrige ce défaut [35], et faire beaucoup de petits pas plutôt que beaucoup d'itérations améliore encore la précision [36].

Pour une chaîne comme une corde, des méthodes spécialisées existent : une projection de proche en proche le long des cheveux ou des cordes [37], ou un solveur direct qui exploite la structure en chaîne (chaque maillon ne touche que ses deux voisins) et résout le système en un temps proportionnel au nombre de maillons [38].

5.3 Systèmes de particules et fluides

Pour un liquide, la méthode de référence en simulation est la *Smoothed Particle Hydrodynamics* (SPH), qui représente le fluide par des particules portant pression et viscosité [39]. Sa variante basée sur les positions rend le fluide incompressible avec la même approche que les contraintes ci-dessus [40]. Ces méthodes demandent des milliers de particules pour un écoulement réaliste.

Un exercice de cours n'a pas besoin d'autant. Dans le jeu du seau, la question est de savoir *quand* l'eau quitte le fond du seau, pas comment elle s'écoule ensuite. Une dizaine de gouttes, traitées comme des particules qui ne peuvent pas traverser les parois, suffisent à montrer ce moment.

5.4 Précision, stabilité et validation

Ces méthodes visent d'abord le réalisme visuel : en jeu vidéo, une corde doit avoir l'air d'une corde. Dans un cours, l'exigence est plus forte : la simulation doit redonner les résultats théoriques quand les hypothèses du cours sont réunies. Il faut donc la valider de trois façons :

— comparer la simulation à la solution exacte quand elle existe (tir parabolique, petites oscillations du pendule) ;

— vérifier les grandeurs conservées, comme l'énergie mécanique sans frottement ;

— réduire le pas et vérifier que le résultat converge vers la même valeur.

L'écart restant doit être mesuré et expliqué, pas caché.

5.5 Afficher : Canvas, SVG ou WebGL

Un navigateur offre trois façons de dessiner. SVG décrit la figure comme un ensemble d'objets (lignes, cercles, textes) que le navigateur garde en mémoire : c'est pratique pour une figure statique, mais chaque objet modifié à chaque image a un coût, ce qui devient sensible avec des centaines d'éléments. Canvas 2D dessine directement des pixels avec des commandes simples [41] ; il suffit pour des vecteurs, des trajectoires et quelques centaines de particules, et la fonction `requestAnimationFrame` cale le dessin sur le rafraîchissement de l'écran. WebGL donne accès à la carte graphique, ce qui n'est utile que pour la 3D ou de très nombreux objets.

5.6 Bibliothèques JavaScript

Des bibliothèques simplifient ce travail. D3 construit des visualisations de données [42], p5.js offre un cadre de dessin pensé pour l'apprentissage de la programmation [43], Three.js gère la 3D [44], et Matter.js est un moteur physique 2D complet [45]. Des bibliothèques comme Chart.js ou Plotly tracent des graphes scientifiques prêts à l'emploi.

Deux raisons nous ont fait écrire les calculs et l'affichage nous-mêmes, en JavaScript sans dépendance. Un moteur physique calcule le mouvement à la place du code de l'exercice, avec ses propres approximations : on ne peut plus montrer à l'étudiant d'où vient le résultat, ni vérifier facilement qu'il est juste. Ensuite, chaque dépendance pose un problème de durée : le site doit encore fonctionner dans plusieurs années, sans maintenance, et s'ouvrir même sans connexion. Pour les graphes, le besoin est étroit (quelques courbes en fonction du temps, avec un curseur synchronisé sur la simulation), et un petit traceur écrit pour le site y suffit.

Chapitre 6

Intelligence artificielle et développement logiciel

Ce chapitre fait le point sur les assistants de programmation fondés sur des modèles de langage : ce qu'ils savent faire, ce que l'on a mesuré de leur apport, et leurs limites. Il se termine par la synthèse de l'état de l'art.

6.1 Assistants de programmation fondés sur des modèles de langage

Les assistants de programmation reposent sur des modèles de langage entraînés sur de grandes quantités de texte et de code. Codex, à l'origine de GitHub Copilot, a montré dès 2021 qu'un tel modèle pouvait écrire une fonction correcte à partir de sa description, sur un jeu d'exercices dont chaque solution est vérifiée par des tests [5].

Les premiers outils complétaient la ligne ou la fonction en cours d'écriture. Les assistants actuels, comme Claude Code ou ChatGPT Codex que nous avons utilisés, sont des *agents* : ils lisent un dépôt entier, modifient plusieurs fichiers, lancent les tests et corrigent leurs erreurs avant de rendre compte. Ils peuvent aussi confier une sous-tâche à une autre instance, lancée dans un contexte séparé. Le banc d'essai SWE-bench mesure cette capacité sur de vrais tickets de projets à code ouvert ; à sa publication, en 2023, le meilleur modèle n'en résolvait qu'environ 2 % [46] [À COMPLÉTER : score actuel sur SWE-bench Verified, avec sa source]. Ces outils évoluent vite, ce qui rend toute mesure de leur apport datée.

6.2 Les gains mesurés

Les mesures de gain se contredisent, et la contradiction est instructive. Dans une expérience contrôlée, des développeurs devaient écrire un petit serveur web en JavaScript : ceux qui disposaient de Copilot ont terminé 55,8 % plus vite que les autres [47]. À l'inverse, une étude randomisée de METR, en 2025, a suivi seize développeurs expérimentés sur 246 tâches réelles dans des projets qu'ils connaissaient bien : avec l'IA, ils ont mis 19 % de temps en plus, alors qu'ils pensaient avoir gagné environ 20 % [48]. Vaithilingam et al. ont aussi observé que Copilot n'améliorait pas toujours le temps ni le taux de réussite, bien que la plupart des participants préfèrent l'utiliser : le temps gagné à écrire est en partie perdu à comprendre et corriger le code proposé [49].

Ces études diffèrent par l'année, l'outil, les participants et la tâche, et ne permettent pas de trancher. Une explication possible est que le gain dépend de la tâche : grand pour du code neuf et bien délimité, plus faible dans un code existant où le développeur connaît déjà la solution. Une leçon, en revanche, est sûre : l'impression de gain ne suffit pas, il faut le mesurer.

Pour ce mémoire, cela veut dire relever, tâche par tâche, ce que l'assistant a fait, ce qu'il a fallu corriger et combien de temps cela a pris.

6.3 Fiabilité, vérification et responsabilité

Un modèle de langage produit du code plausible, pas du code vérifié. Pearce et al. ont fait générer environ 1700 programmes par Copilot dans des situations à risque : environ 40 % contenaient une faille de sécurité [9]. En physique, l'équivalent d'une faille est une formule fausse qui s'affiche avec assurance, un signe inversé ou une hypothèse ajoutée sans le dire. L'erreur est d'autant plus dangereuse qu'une animation fausse peut avoir l'air juste.

La réponse habituelle du génie logiciel s'applique ici : des tests automatiques, une relecture humaine, et une vérification séparée de ce que l'outil affirme. Si cette vérification est confiée au même modèle, même lancé sans l'historique de la session, elle n'est pas pleinement indépendante : les deux peuvent partager la même erreur. La responsabilité de l'exactitude reste donc celle des auteurs, qui doivent comprendre et pouvoir justifier chaque formule affichée.

6.4 Usages en enseignement

La littérature sur l'IA en éducation porte surtout sur l'usage par les étudiants : aide à la rédaction, tutorat, risques pour l'évaluation et l'esprit critique [50]. L'usage étudié ici est différent : l'IA sert aux enseignants pour produire des ressources, et les étudiants n'interagissent qu'avec le résultat, vérifié. Nous n'avons pas trouvé d'étude qui mesure cet usage ; c'est l'objet de la deuxième question de ce mémoire.

6.5 Synthèse de l'état de l'art : exigences pour le site

Les cinq chapitres de cette partie donnent des exigences précises pour nos animations. Le tableau 6.1 les rassemble, avec leur origine et la façon dont le site y répond ; la partie Réalisation les détaille.

Tableau 6.1 – Exigences tirées de l'état de l'art et réponse du site.

	Exigence (origine)	Réponse du site
E1	Montrer toutes les forces, la vitesse et l'accélération séparément (chapitre 2)	vecteurs dans la scène et diagramme du corps libre toujours visible, une couleur fixe par type de force
E2	Parler la langue du cours (chapitre 2)	notations du formulaire, formules écrites avec leurs valeurs substituées
E3	Laisser prédire puis vérifier, avec un guidage (chapitre 3)	verdict explicite, hypothèses du modèle écrites sur la page, défis du mode jeu
E4	Relier les représentations sans surcharger (chapitre 3)	scène, formules et graphes synchronisés sur le même instant, sur un seul écran quand c'est possible
E5	Un jeu dont la règle est la physique (chapitre 4)	conditions de réussite égales aux relations du cours, réponses masquées, défis tirés au hasard
E6	Rester juste quand on intègre (chapitre 5)	formule exacte quand elle existe, RK4 ou contraintes sinon, écart à la théorie mesuré
E7	Ouvrir partout et durer (chapitre 5)	HTML, CSS et JavaScript sans dépendance, pages ouvrables sans serveur, testées sur téléphone, tablette et ordinateur
E8	Vérifier ce que l'IA produit et mesurer son apport (chapitre 6)	tests automatiques, vérification séparée de la physique, relecture humaine, journal des tâches

Deuxième partie

Réalisation

Chapitre 7

Cahier des charges et choix techniques

7.1 Public visé et contraintes pédagogiques

7.2 Exigences fonctionnelles

7.3 Exigences techniques

7.4 Choix technologiques

Chapitre 8

Architecture du site

8.1 Organisation du projet

8.2 Séparation calcul et affichage

8.3 Éléments visuels partagés

8.4 Mise en page et adaptation aux écrans

Chapitre 9

Exercices réalisés

9.1 Mouvement d'un projectile

9.2 Pendule

9.3 Mouvement circulaire uniforme

9.4 Looping

9.5 Looping avec wagon : cercle et clothoïde

Chapitre 10

Mode jeu : le seau d'eau

10.1 Principe et lien avec le looping

10.2 Modèle physique

10.3 Résolution numérique

10.4 Pilotage par le moteur

10.5 Défis toujours solubles

Chapitre 11

Méthodologie de développement assistée par IA

11.1 Cadre et règles du projet

11.2 Cycle de développement

11.3 Agents de test et de vérification

11.4 Rôle de l'enseignant

Chapitre 12

Validation

12.1 Vérification de la physique

12.2 Tests unitaires

12.3 Tests d'intégration sur plusieurs appareils

12.4 Erreurs détectées et corrigées

Troisième partie

Résultats

Chapitre 13

Protocole d'évaluation

13.1 Questions évaluées

13.2 Participants et déroulement

13.3 Instruments de mesure

13.4 Analyse des données

13.5 Aspects éthiques

Chapitre 14

Résultats quantitatifs

14.1 Gains d'apprentissage

14.2 Utilisation du site

14.3 Mode jeu

Chapitre 15

Résultats qualitatifs

15.1 Retours des étudiants

15.2 Retours des enseignants

15.3 Observations en classe

Chapitre 16

Discussion

16.1 Réponses aux questions de recherche

16.2 Liens avec la littérature

16.3 Limites

16.4 Éthique, accessibilité et durabilité

16.5 Recommandations

Chapitre 17

Conclusion et perspectives

17.1 Synthèse des contributions

17.2 Perspectives

Bibliographie

[1] I. A. Halloun and D. Hestenes, "Common sense concepts about motion", *American Journal of Physics*, vol. 53, no. 11, pp. 1056–1065, 1985. DOI: 10.1119/1.14031

[2] R. R. Hake, "Interactive-engagement versus traditional methods: a six-thousand-student survey of mechanics test data for introductory physics courses", *American Journal of Physics*, vol. 66, no. 1, pp. 64–74, 1998. DOI: 10.1119/1.18809

[3] C. E. Wieman, W. K. Adams, and K. K. Perkins, "PhET: simulations that enhance learning", *Science*, vol. 322, no. 5902, pp. 682–683, 2008. DOI: 10.1126/science.1161948

[4] N. Rutten, W. R. van Joolingen, and J. T. van der Veen, "The learning effects of computer simulations in science education", *Computers & Education*, vol. 58, no. 1, pp. 136–153, 2012. DOI: 10.1016/j.compedu.2011.07.017

[5] M. Chen, J. Tworek, H. Jun, et al., *Evaluating large language models trained on code*, 2021. arXiv: 2107.03374.

[6] V. Legat, *Master thesis proposal: AI-assisted development of interactive JavaScript animations for introductory physics education*, UCLouvain, École polytechnique de Louvain, 2026. Accessed: Nov. 5, 2026. [Online]. Available: https://perso.uclouvain.be/vincent.legat/documents/memo2990/master_thesis_proposal_intro_physics_animations_ai.html

[7] École polytechnique de Louvain, *Formulaire du cours de Physique 1 (EPL1201)*, Document de cours, Fichier pdfassets/epl1201-formulaire.pdf du dépôt.

[8] V. Legat, J. Lambrechts, I. Deleersnijder et C. Dubois, *Physique 1 (LEPL1201) : animations*, Site du cours, UCLouvain, 2020. visité le 5 nov. 2026. adresse : https://perso.uclouvain.be/vincent.legat/zouLab/epl1201.php?action=animations

[9] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions", in *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 754–768. DOI: 10.1109/SP46214.2022.9833571

[10] "Physics simulator", Accessed: Nov. 5, 2026. [Online]. Available: https://physicsviz.github.io/simulator/

[11] L. Viennot, "Spontaneous reasoning in elementary dynamics", *European Journal of Science Education*, vol. 1, no. 2, pp. 205–221, 1979. DOI: 10.1080/0140528790010209

[12] A.-M. Pendrill, "Rollercoaster loop shapes", *Physics Education*, vol. 40, no. 6, pp. 517–521, 2005. DOI: 10.1088/0031-9120/40/6/001

[13] D. Hestenes, M. Wells, and G. Swackhamer, "Force concept inventory", *The Physics Teacher*, vol. 30, no. 3, pp. 141–158, 1992. DOI: 10.1119/1.2343497

[14] S. Freeman et al., "Active learning increases student performance in science, engineering, and mathematics", *Proceedings of the National Academy of Sciences*, vol. 111, no. 23, pp. 8410–8415, 2014. DOI: 10.1073/pnas.1319030111

[15] W. K. Adams et al., “A study of educational simulations part I – engagement and learning”, *Journal of Interactive Learning Research*, vol. 19, no. 3, pp. 397–419, 2008.

[16] N. D. Finkelstein et al., “When learning about the real world is better done virtually: a study of substituting computer simulations for laboratory equipment”, *Physical Review Special Topics – Physics Education Research*, vol. 1, no. 1, p. 010 103, 2005. DOI: 10.1103/PhysRevSTPER.1.010103

[17] T. de Jong and W. R. van Joolingen, “Scientific discovery learning with computer simulations of conceptual domains”, *Review of Educational Research*, vol. 68, no. 2, pp. 179–201, 1998. DOI: 10.3102/00346543068002179

[18] J. Sweller, “Cognitive load during problem solving: effects on learning”, *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988. DOI: 10.1207/s15516709cog1202_4

[19] R. E. Mayer, *Multimedia Learning*, 2nd ed. Cambridge: Cambridge University Press, 2009.

[20] S. Ainsworth, “DeFT: a conceptual framework for considering learning with multiple representations”, *Learning and Instruction*, vol. 16, no. 3, pp. 183–198, 2006. DOI: 10.1016/j.learninstruc.2006.03.001

[21] PhET Interactive Simulations, *PhET interactive simulations*, University of Colorado Boulder. Accessed: Nov. 5, 2026. [Online]. Available: https://phet.colorado.edu

[22] F. Esquembre, “Easy Java simulations: a software tool to create scientific simulations in Java”, *Computer Physics Communications*, vol. 156, no. 2, pp. 199–204, 2004. DOI: 10.1016/S0010-4655(03)00440-5

[23] E. Neumann, *myPhysicsLab – physics simulation with JavaScript*. Accessed: Nov. 5, 2026. [Online]. Available: https://www.myphysicslab.com

[24] GeoGebra, *GeoGebra – dynamic mathematics for everyone*. Accessed: Nov. 5, 2026. [Online]. Available: https://www.geogebra.org

[25] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, “From game design elements to gamefulness: defining “gamification””, in *Proceedings of the 15th International Academic MindTrek Conference*, 2011, pp. 9–15. DOI: 10.1145/2181037.2181040

[26] J. Hamari, J. Koivisto, and H. Sarsa, “Does gamification work? – a literature review of empirical studies on gamification”, in *47th Hawaii International Conference on System Sciences*, 2014, pp. 3025–3034. DOI: 10.1109/HICSS.2014.377

[27] M. Sailer and L. Homner, “The gamification of learning: a meta-analysis”, *Educational Psychology Review*, vol. 32, pp. 77–112, 2020. DOI: 10.1007/s10648-019-09498-w

[28] J. C. Butcher, *Numerical Methods for Ordinary Differential Equations*, 3rd ed. Chichester: Wiley, 2016.

[29] E. Hairer, C. Lubich, and G. Wanner, *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, 2nd ed. Berlin: Springer, 2006.

[30] L. Verlet, “Computer “experiments” on classical fluids. I. thermodynamical properties of Lennard-Jones molecules”, *Physical Review*, vol. 159, no. 1, pp. 98–103, 1967. DOI: 10.1103/PhysRev.159.98

[31] T. Jakobsen, *Advanced character physics*, Game Developers Conference, 2001.

[32] G. Fiedler, *Fix your timestep!* Gaffer On Games, 2004. Accessed: Nov. 5, 2026. [Online]. Available: https://gafferongames.com/post/fix_your_timestep/

[33] M. Müller, B. Heidelberger, M. Hennix, and J. Ratcliff, “Position based dynamics”, *Journal of Visual Communication and Image Representation*, vol. 18, no. 2, pp. 109–118, 2007. DOI: 10.1016/j.jvcir.2007.01.005

[34] J. Bender, M. Müller, M. A. Otaduy, M. Teschner, and M. Macklin, "A survey on position-based simulation methods in computer graphics", *Computer Graphics Forum*, vol. 33, no. 6, pp. 228–251, 2014. DOI: 10.1111/cgf.12346

[35] M. Macklin, M. Müller, and N. Chentanez, "XPBD: position-based simulation of compliant constrained dynamics", in *Proceedings of the 9th International Conference on Motion in Games*, 2016, pp. 49–54. DOI: 10.1145/2994258.2994272

[36] M. Macklin et al., "Small steps in physics simulation", in *Proceedings of the 18th Annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, 2019. DOI: 10.1145/3309486.3340247

[37] M. Müller, T.-Y. Kim, and N. Chentanez, "Fast simulation of inextensible hair and fur", in *Workshop on Virtual Reality Interaction and Physical Simulation (VRIPHYS)*, The Eurographics Association, 2012. DOI: 10.2312/PE/vriphys/vriphys12/039-044

[38] C. Deul, T. Kugelstadt, M. Weiler, and J. Bender, "Direct position-based solver for stiff rods", *Computer Graphics Forum*, vol. 37, no. 6, pp. 313–324, 2018. DOI: 10.1111/cgf.13326

[39] J. J. Monaghan, "Smoothed particle hydrodynamics", *Annual Review of Astronomy and Astrophysics*, vol. 30, pp. 543–574, 1992. DOI: 10.1146/annurev.aa.30.090192.002551

[40] M. Macklin and M. Müller, "Position based fluids", *ACM Transactions on Graphics*, vol. 32, no. 4, 104:1–104:12, 2013. DOI: 10.1145/2461912.2461984

[41] WHATWG, *HTML living standard – the canvas element*. Accessed: Nov. 5, 2026. [Online]. Available: https://html.spec.whatwg.org/multipage/canvas.html

[42] M. Bostock, V. Ogievetsky, and J. Heer, "D^3: data-driven documents", *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, pp. 2301–2309, 2011. DOI: 10.1109/TVCG.2011.185

[43] Processing Foundation, *P5.js*. Accessed: Nov. 5, 2026. [Online]. Available: https://p5js.org

[44] three.js authors, *Three.js – JavaScript 3d library*. Accessed: Nov. 5, 2026. [Online]. Available: https://threejs.org

[45] L. Brummitt, *Matter.js – a 2D rigid body physics engine for the web*. Accessed: Nov. 5, 2026. [Online]. Available: https://brm.io/matter-js/

[46] C. E. Jimenez et al., "SWE-bench: can language models resolve real-world GitHub issues?", in *International Conference on Learning Representations (ICLR)*, 2024. arXiv: 2310.06770.

[47] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, *The impact of AI on developer productivity: evidence from GitHub Copilot*, 2023. arXiv: 2302.06590.

[48] J. Becker, N. Rush, E. Barnes, and D. Rein, "Measuring the impact of early-2025 AI on experienced open-source developer productivity", METR, Tech. Rep., 2025. [Online]. Available: https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/

[49] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: evaluating the usability of code generation tools powered by large language models", in *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, 2022, pp. 1–7. DOI: 10.1145/3491101.3519665

[50] E. Kasneci, K. Sessler, S. Küchemann, et al., "ChatGPT for good? on opportunities and challenges of large language models for education", *Learning and Individual Differences*, vol. 103, p. 102 274, 2023. DOI: 10.1016/j.lindif.2023.102274

Annexe A

Formulaire du cours

Annexe B

Questionnaires

Annexe C

Liste des exercices et du code

Annexe D

Détails des tests

UNIVERSITÉ CATHOLIQUE DE LOUVAIN
École polytechnique de Louvain
Rue Archimède, 1 bte L6.11.01, 1348 Louvain-la-Neuve, Belgique | **www.uclouvain.be/epl**